TLS — Fase 1: Logika Penyelesaian Masalah

Nama : Bandang Raditya Maheswara
Kelompok : Obsidian

Problem 1 — Eliminasi Astronot dalam Lingkaran

*Input: jumlah astronot N dan nilai awal K. Output: urutan seluruh astronot yang dieliminasi dan nomor astronot yang bertahan. Logika disajikan dalam bentuk **pseudocode**.*

Pseudocode

```
ALGORITMA EliminasiAstronot

KAMUS
  N, K, sisa, current, target, jumlahLangkah : integer
  tereliminasi : array [1..N] of boolean     { true = sudah keluar dari lingkaran }

DESKRIPSI
  INPUT N, K
  JIKA N < 1 MAKA
      OUTPUT "Input tidak valid"
      BERHENTI
  AKHIR JIKA
  JIKA K < 2 MAKA K ← 2                      { aturan: K minimal 2 }

  UNTUK i ← 1 SAMPAI N LAKUKAN
      tereliminasi[i] ← false
  AKHIR UNTUK

  current ← 1                                { hitungan pertama mulai dari astronot 1 }
  sisa ← N

  SELAMA sisa > 1 LAKUKAN
      target ← current                       { current = hitungan ke-1 }
      jumlahLangkah ← (K - 1) MOD sisa
      ULANGI jumlahLangkah KALI
          target ← astronotAktifBerikutnya(target)
      AKHIR ULANGI

      OUTPUT target                          { astronot yang dieliminasi }
      tereliminasi[target] ← true
      sisa ← sisa - 1

      JIKA target MOD 2 = 0 MAKA             { nomor genap }
          K ← K + 2
      SELAIN ITU                             { nomor ganjil }
          K ← K - 1
      AKHIR JIKA
      JIKA K < 2 MAKA K ← 2

      current ← astronotAktifBerikutnya(target)   { putaran berikut mulai di sini }
  AKHIR SELAMA

  OUTPUT "Astronot yang bertahan:", current
AKHIR ALGORITMA

FUNGSI astronotAktifBerikutnya(dari : integer) → integer
  idx ← dari
  ULANGI
      idx ← (idx MOD N) + 1                  { setelah N kembali ke 1 }
  SAMPAI tereliminasi[idx] = false
  KEMBALIKAN idx
AKHIR FUNGSI
```

Penjelasan Logika

Masalah ini merupakan variasi dari *Josephus Problem*, yaitu eliminasi berulang pada orang-orang yang berdiri melingkar. Lingkaran astronot direpresentasikan dengan array bernomor 1 sampai N beserta penanda `tereliminasi[i]` yang menunjukkan apakah astronot ke-i sudah keluar. Variabel `current` menyimpan astronot tempat hitungan dimulai (awalnya astronot nomor 1) dan dihitung sebagai hitungan ke-1, sehingga astronot yang mendapat hitungan ke-K ditemukan dengan menggeser sebanyak (K − 1) langkah ke astronot aktif berikutnya. Fungsi `astronotAktifBerikutnya` melompati astronot yang sudah tereliminasi dan kembali ke nomor 1 setelah nomor N, sehingga sifat melingkar tetap terjaga. Karena lingkaran hanya berisi `sisa` astronot aktif, mengelilingi lingkaran penuh tidak mengubah hasil, sehingga cukup digeser (K − 1) MOD `sisa` langkah agar perulangan tetap singkat meskipun K besar.

Setelah astronot `target` ditemukan, nomornya dicetak sebagai bagian dari urutan eliminasi, ditandai tereliminasi, dan nilai `sisa` dikurangi satu. Nilai K kemudian diperbarui berdasarkan nomor astronot yang baru dieliminasi tersebut: jika genap maka K bertambah 2, jika ganjil maka K berkurang 1, dan jika hasilnya kurang dari 2 maka K dipaksa menjadi 2. Nilai K yang baru ini baru dipakai pada putaran berikutnya. Putaran berikutnya dimulai dari astronot yang berada tepat setelah astronot yang dieliminasi, sehingga `current` diperbarui dengan memanggil `astronotAktifBerikutnya` pada `target`.

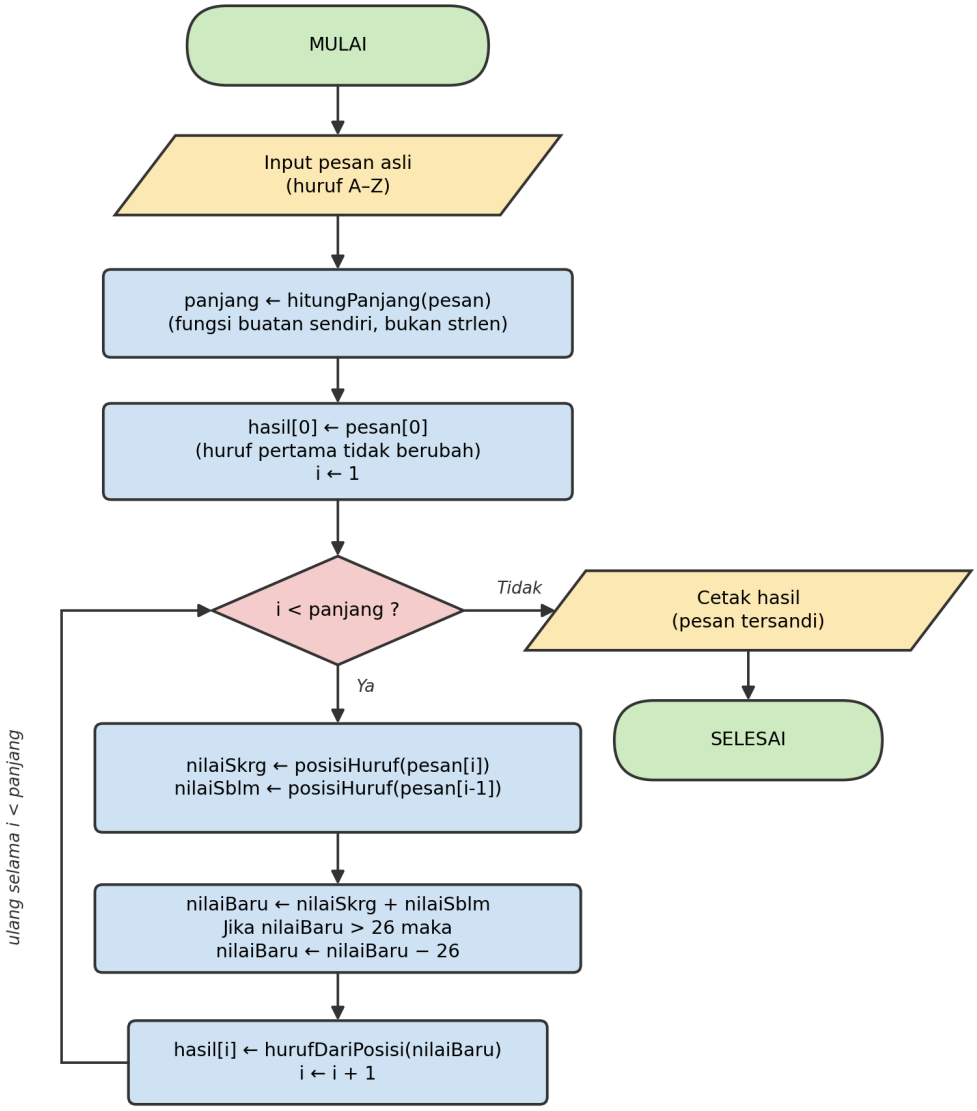
Perulangan `SELAMA sisa > 1` berhenti ketika hanya tersisa satu astronot, dan astronot itulah yang bertahan; pada saat itu `current` sudah menunjuk astronot tersebut. Sebagai contoh, untuk N = 5 dan K awal = 2, nilai K pada empat putaran berturut-turut adalah 2, 4, 3, dan 2, sehingga urutan eliminasinya 2, 1, 5, 4 dan astronot yang bertahan adalah nomor 3. Kasus N = 1 tertangani dengan sendirinya karena perulangan tidak dijalankan dan astronot nomor 1 langsung menjadi yang bertahan.

Problem 2 — Sistem Sandi Pergeseran Huruf

*Input: pesan asli (huruf A–Z). Output: pesan yang telah disandikan. Logika disajikan dalam bentuk **flowchart**.*

Flowchart

Flowchart Problem 2 — Sistem Sandi Pergeseran Huruf



Keterangan: oval hijau = mulai/selesai · jajar genjang kuning = input/output · persegi biru = proses · belah ketupat merah = keputusan

Penjelasan Logika

Sistem sandi ini berbeda dari sandi Caesar biasa karena besar pergeserannya tidak tetap, melainkan bergantung pada huruf sebelumnya. Setiap huruf diubah menjadi angka sesuai posisinya dalam alfabet (A = 1 sampai Z = 26) oleh fungsi `posisiHuruf`. Huruf pertama tidak memiliki huruf sebelumnya sehingga langsung disalin ke hasil. Untuk huruf ke-i berikutnya, nilai barunya adalah nilai huruf tersebut ditambah nilai huruf ke-(i − 1). Yang dipakai sebagai pergeseran adalah huruf pada pesan asli, bukan huruf hasil sandi; pada contoh ALIENS, huruf I digeser 12 (nilai L pada pesan asli), bukan 13 (nilai M, hasil sandi dari L), sehingga 9 + 12 = 21 yaitu huruf U.

Hasil penjumlahan dapat melebihi 26, sehingga diperlukan penanganan putar balik: jika `nilaiBaru > 26` maka dikurangi 26 agar hitungan kembali dimulai dari A. Nilai terbesar yang mungkin adalah 26 + 26 = 52, sehingga satu kali pengurangan sudah cukup. Misalnya pada huruf S: 19 + 14 = 33, lalu 33 − 26 = 7 yang berarti huruf G. Nilai akhir (1–26) diubah kembali menjadi huruf oleh fungsi `hurufDariPosisi` dan disimpan pada array `hasil`. Perulangan berjalan sekali dari indeks 1 (huruf kedua) sampai huruf terakhir, sehingga pesan ALIENS menghasilkan AMUNSG.

Pesan diasumsikan hanya berisi huruf seperti pada contoh soal; huruf kecil diubah menjadi huruf besar sebelum diproses dan karakter selain huruf ditolak. Panjang pesan dihitung dengan fungsi `hitungPanjang` yang dibuat sendiri sebagai pengganti `strlen`, sehingga seluruh logika hanya memakai perulangan, percabangan, dan operasi aritmetika dasar tanpa function dari library.